Test strategy: Feedback Intelligence Platform

Status	Approved
Version	1.1
Date	2026-11-05
Owner	Benjamin Rose
Related	Requirements, Design, Data dictionary

This document explains how the platform is tested: what each kind of test proves, where it runs, and which risks it covers. The aim is that a change can't reach the live dashboard unless the tests say it's safe, and that the few things checked by hand are named rather than assumed.

1. Principles

1. **Test at the cheapest level that can prove the point.** Business rules are tested with fakes in milliseconds; only behavior that depends on SQL Server, Presidio, or file formats uses real containers.
2. **Privacy is tested end to end, not just in parts.** A redactor that works in isolation proves nothing if a later step publishes the original text, so the PII test set runs through the whole pipeline (section 4).
3. **What CI checks is also checked at run time where it can change.** CI proves the code is right; the live redaction service, the data, and the config can still go wrong on the day, so the pipeline re-checks those before it publishes (section 5).
4. **A failing test explains itself.** Messages name the rule, the file, or the kind of value that failed, never personal data.

2. Test levels

Level	Project or folder	What it proves	Speed	Runs
Unit	`tests/FeedbackIntel.Core.Tests`	Each pipeline step and rule in Core, with fakes for every interface: the orchestrator stops on the first failure, ingestion skips duplicates and honors watermarks and backfills, retries back off correctly, settings reject out-of-range values	Seconds	CI, every pull request
Integration	`tests/FeedbackIntel.Integration.Tests`	The Infrastructure adapters against real services in containers (Testcontainers): migrations, every SQL store and the reporting procedure, Presidio redaction, Parquet history, publishing, GitHub retry handling	A few minutes	CI, every pull request

Level	Project or folder	What it proves	Speed	Runs
Contract	`Integration.Tests/Contract`	Every published and history file contains exactly the columns listed in section 7 of the data dictionary, read from the document itself, so the code and the contract can't drift apart (NFR-PRIV-02)	Seconds, after a pipeline run in the test	CI
End-to-end privacy	`Integration.Tests/Privacy/EndToEndPiiTests.cs`	Issues built from the PII test set go through redaction, the cleaned layer, reporting, publishing, and history, and no known value appears in any output file (FR-PRV-04, NFR-PRIV-01)	About a minute	CI
Config	`Core.Tests/Configuration/ConfigFileTests.cs`	The real `config/pipeline.json` loads with the pipeline's own settings classes, has no unknown or misspelled settings, and passes every range check (FR-CFG-02). Runs first as its own CI step	Seconds	CI
Dashboard	`dashboard/tests`	The dashboard's SQL runs in the Node build of the same DuckDB package the browser uses: filters combine correctly, date ranges include both days, search treats input as text, never SQL; plus the wording and formatting helpers	Seconds	CI, "Dashboard tests" job
Build checks	CI workflow	No tracked file contains a secret (detect-secrets, FR-DEV-01); the solution builds with warnings as errors; C# formatting matches `.editorconfig` and dashboard code passes Prettier (NFR-MNT-04); NuGet and npm packages have no known high-severity vulnerabilities (NFR-SEC-04); and the dashboard build can still download and assemble every pinned library	Seconds	CI

Coverage

C# line coverage must stay at or above **80%** (NFR-MNT-01). CI measures it across both test projects and fails the "Build and test" check below the minimum. The `Pipeline` project, which is mostly wiring, isn't loaded by the tests, so it isn't part of the measurement; every daily run exercises it.

SQL

Every SQL transform has a test that checks its output against expected values (NFR-MNT-02): the reporting procedure (`ReportingAndPublishingTests`), the cleaned-layer and watermark upserts, the quality checks (one test per kind of broken data), and the run store. The tests run against the same pinned SQL Server image the pipeline uses, so a query that works only on another version fails here first.

3. Fakes and real services

Dependency	Unit tests	Integration tests
GitHub API	Fake `IIssueSource`	Octokit objects built in the test for the conversion code, and simulated errors for retries; the real API is used only by the daily run
SQL Server	Fake stores	Real container, a new database per test
Presidio	Fake `IPiiRedactor`	Real analyzer and anonymizer containers
Clock	`FakeClock` with fixed times	Fixed times passed in
File system	Not used	Temporary folders, deleted after each test

Interfaces for every external service (NFR-MNT-03) are what make this split possible: each Core step is tested without a network, and each adapter is tested once against the real thing.

4. The PII test set

`src/FeedbackIntel.Core/Privacy/PiiTestSet.cs` holds made-up issue text with one known value for each kind of personal information the redactor must remove: emails, IPv4 and IPv6 addresses, classic and fine-grained GitHub tokens, and usernames in Windows, Linux, and macOS file paths. The tokens are built at run time, so nothing in the repository looks like a real secret.

It is used in three places, so a sample added once is checked everywhere:

1. **`PiiRedactionTests`** checks that the redaction service replaces each value with the right placeholder, and leaves ordinary issue text unchanged.
2. **`EndToEndPiiTests`** runs the full pipeline over issues made from the samples and searches every value in every output file for the known values. A new Parquet file fails the test until it is added to the search, so nothing can be published unscanned.
3. **The privacy check step** sends the samples to the live redaction service at the start of every pipeline run (section 5).

When a real issue reveals a missed pattern, the fix starts with a new sample (US-18).

5. Checks at run time

These run inside every pipeline run, because what they check can change without a code change.

Check	When	On failure
Config validation	Start-up	The run stops before touching anything
Freshness check	After history is restored	Reports stale data to the alert job; the run continues
Raw issue validation	During ingestion	Bad issues are rejected and counted; too many rejections stop the run

Check	When	On failure
Privacy check	Before redaction	The run stops; nothing is redacted or published
Quality checks (14)	Before publishing	The run stops; the dashboard keeps the last good data

Any failure opens or updates a `pipeline-alert` issue (FR-OPS-02).

6. Checked by hand

Some things are cheaper to check by eye than to automate at this size. They are checked before each milestone is marked done, and after any change to the dashboard.

What	How
Dashboard pages in a browser	Open Overview, Trends, and Issues on the live site; try each filter; check phone width and dark mode
Dashboard load time (NFR-PERF-02)	Measured with throttled network when the dashboard was built; recorded in design section 6
Redaction on real issues (assumption A5)	Search the published titles for patterns the test set covers; read the redacted titles for over-redaction. Results are in the project plan
Alert issues	Read the issue the first time each kind of failure happens

A browser test suite for the dashboard is not planned: the query logic, which is where bugs would matter most, is already tested in Node with the real engine.

7. Where tests run

Workflow	Trigger	Jobs and required checks
`ci.yml`	Every pull request and every push to `main`	**Build and test** (required by the branch ruleset): build, formatting, config validation, unit and integration tests, coverage. **Dashboard tests**: install, test, and build the dashboard
`pipeline.yml`	Daily schedule and manual runs	The run-time checks in section 5

Merges to `main` go through pull requests only, and a pull request can't merge until the required checks pass.

8. Requirements covered

Requirement	Tested by
FR-ING-02, FR-ING-06	`IngestIssuesStepTests`, `SqlWatermarkStoreTests`
FR-ING-04	`RetryPolicyTests`, `GitHubRetryTests`, `PresidioRetryTests`
FR-PRV-01 to 04, NFR-PRIV-01	`PiiRedactionTests`, `EndToEndPiiTests`, `PrivacyCheckStepTests`, `RedactIssuesStepTests`
FR-DAT-02, FR-DAT-06	`ParquetHistoryStoreTests`, `HistoryStepTests`
FR-DAT-05	`SqlQualityCheckerTests`, `QualityChecksStepTests`
FR-OPS-02, FR-OPS-03	`FreshnessCheckStepTests`, `GitHubActionsOutputTests`, `SqlErrorDescriberTests`; the alert job's script is checked by hand

Requirement	Tested by
FR-CFG-02	`ConfigFileTests`, `PipelineSettingsTests`, `BackfillOptionsTests`
FR-DSH-03, NFR-PERF-01	`dashboard/tests/queries.test.mjs`
NFR-PRIV-02	`DataContractTests`
NFR-MNT-01, NFR-MNT-04	CI coverage gate, `dotnet format`, and Prettier checks
FR-DEV-01, NFR-SEC-01	CI secret scan
NFR-SEC-04	NuGet audit during the build, and `npm audit`

Revision history

Version	Date	Change
1.1	2026-11-05	Added the secret scan, npm audit, and Prettier checks from the M3 review.
1.0	2026-11-05	First version, written at the end of M3.